



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт перспективных технологий и индустриального программирования (ИПТИП)
Кафедра индустриального программирования

ОТЧЁТ ПО УЧЕБНОЙ ПРАКТИКЕ
Технологическая (проектно-технологическая) практика

приказ Университета о направлении на практику от «29» августа 2025 г.
№ 7136-С

Отчет представлен к
рассмотрению:

Студент группы ЭФБО-06-22 «20» _декабря_ 2025 г. _____ Холбоев Б.У.
(подпись)

Отчет утвержден.
Допущен к защите:

Руководитель практики от
кафедры

«20» _декабря_ 2025 г. _____ Клесов Д.Н.
(подпись)

Руководитель практики от
профильной организации

«20» _декабря_ 2025 г. _____ (Насимов Р.Р.)
(подпись)

Москва 2025



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт перспективных технологий и индустриального программирования (ИПТИП)
Кафедра индустриального программирования

ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ
Технологическая (проектно-технологическая) практика
Студенту 4 курса учебной группы ЭФБО-06-22

Студент Холбоев Бахтиёр Уктамович

Место и время практики: РТУ МИРЭА, ИП, с 01 сентября 2025 г. по 20 декабря 2025 г.

Должность на практике: студент

1. СОДЕРЖАНИЕ ПРАКТИКИ:

- 1.1. Изучить: профессиональные стандарты в профессиональной области.
- 1.2. Практически выполнить: задания в профессиональной области от непосредственного руководителя в структуре организации.
- 1.3. Ознакомиться: нормы труда в профессиональной области, локальные нормативные акты в организации, правила охраны труда и должностные инструкции.

2. ДОПОЛНИТЕЛЬНОЕ ЗАДАНИЕ: По итогам выполнения исследования необходимо подготовить отчёт по практике в заданной форме.

3. ОРГАНИЗАЦИОННО-МЕТОДИЧЕСКИЕ УКАЗАНИЯ: В процессе прохождения практики следует загружать результаты выполнения в рабочую область в системе дистанционного обучения.

Руководитель практики от кафедры

«01» __ сентября ____ 2025 г.

(подпись) (Клесов Д.Н.)

Руководитель практики от кафедры

«01» __ сентября ____ 2025 г.

(подпись) (Насимов Р.Р.)

Задание получил

«01» __ сентября ____ 2025 г.

(подпись) (Холбоев Б.У.)

СОГЛАСОВАНО:

Заведующий кафедрой:

«01» __ сентября ____ 2025 г.

(подпись) (Юдин А.В.)



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

**РАБОЧИЙ ГРАФИК ПРОВЕДЕНИЯ
ОЗНАКОМИТЕЛЬНОЙ ПРАКТИКИ**

Студент: Холбоев Б.У. 4 курса группы ЭФБО-06-22 очной формы обучения, обучающегося по направлению подготовки 09.03.02 «Информационные системы и технологии».

Неделя	Сроки выполнения	Этап	Отметка о выполнении
1-4	с 01.09.2025 по 28.09.2025	Подготовительный этап, включающий в себя организационное собрание	выполнено
5-12	с 29.09.2025 по 23.11.2025	Выполнение задания по практике в соответствии с выданным заданием студента	выполнено
13-14	с 24.11.2025 по 07.12.2025	Подготовка отчета по практике	выполнено
15-16	с 08.12.2025 по 20.12.2025	Защита отчета по практике у руководителя практики	выполнено

Руководитель практики от
кафедры _____ / Клесов Д.Н., доцент/

Руководитель практики от
профильной организации _____ / Насимов Р.Р., генеральный директор/

Обучающийся _____ / Холбоев Б.У. /

Согласовано:
Заведующий кафедрой _____ /Юдин А.В./

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ	4
ВВЕДЕНИЕ	5
ПЕРЕЧЕНЬ ВЫПОЛНЕННЫХ РАБОТ	7
1. Реализация моделей Many-to-Many в NestJS + TypeORM	7
2. Механизм автоматической нормализации и проверки данных	9
3. Реализация JWT-аутентификации и механизма refresh-токенов	11
4. React-компонент поиска с дебаунсом	13
5. Docker Compose: поднять три сервиса (Node + PostgreSQL + pgAdmin)	15
ЗАКЛЮЧЕНИЕ	17

ВВЕДЕНИЕ

Современная IT-индустрия стремительно развивается, что очень требует качественной подготовки специалистов в области программной инженерии. Теоретические знания, полученные в процессе обучения, составляют основу профессиональной компетентности, но именно практика позволяет перенести эти знания в реальную производственную среду, ознакомиться со стандартами развития предприятия и овладеть прикладными навыками, необходимыми в отрасли. Практика становится основополагающим этапом в формировании будущего разработчика, обеспечивающим его интеграцию в процессы и инструменты современных компаниях.

Основной целью этой практики было получение опыта участия во всем цикле разработки и обслуживания программных решений. В рамках задачи требовалось решать задачи, связанные с обработкой и аутентификацией данных, оптимизацией производительности серверных компонентов, внедрением механизмов реального времени и настройкой системы контроля доступа. Это позволило нам изучить особенности построения архитектуры корпоративной системы, взаимодействия между микросервисами, организации API и управления данными на уровне базы данных.

В ходе стажировки были освоены и претворены в жизнь основные элементы профессиональной деятельности разработчика: работа с системами контроля версий, участие в процессе проверки кода, следование шаблонам стиля кода, работа в команде для решения сложных технических задач. Реализация функций для массового импорта данных, разработка сервиса для обработки пакетов JSON партнерами, оптимизация производительности отдельных модулей, внедрение веб-уведомлений и создание системы ролей и доступа позволили углубиться в реальные бизнес-задачи и понять принципы создания надежных и масштабируемых приложений.

Накопленный опыт стал важным этапом профессионального роста, который позволяет не только углублять технические компетенции, но и

развивать навыки анализа требований, оценки сложности задач, разработки решений и работы в среде развития команды. Практика позволила нам увидеть работу ИТ-компании изнутри, объединить теоретические знания с реальными производственными процессами и определить направления профессионального развития.

ПЕРЕЧЕНЬ ВЫПОЛНЕННЫХ РАБОТ

1. Реализация моделей Many-to-Many в NestJS + TypeORM

Дата выдачи: 5 сентября 2025 г.

Автор задачи: Насимов Р.Р., руководитель отдела разработки.

Срок для исполнения: 10 рабочих дней.

Система должна обеспечивать управление проектами и пользователями, но текущая архитектура не позволяет связать один объект с несколькими элементами другого, и наоборот. Вам необходимо разработать соответствующие объекты `user` и `project`, установить двустороннюю связь "Many-to-Many" между ними и предоставить пользователям возможность ознакомиться с проектом. Эта функция поможет вам правильно создавать промежуточные таблицы и загружать связанные данные через API.

Описание выполненных работ:

В рамках работы сначала было проанализировано взаимодействие с объектом предметной области "Проект" и сущностью "Пользователь". В зависимости от требований было определено, что одна сущность может относиться ко многим объектам из другой сущности и наоборот, что соответствует соотношению "Many-to-Many".

На втором этапе были созданы два типа объектов: `user` и `project`. Были описаны поля для обеих моделей и добавлены многие декораторы `@ManyToMany`. В сущности пользователя функция `@JoinTable()` определена таким образом, что промежуточные таблицы `user_projects` могут создаваться автоматически без ручного объяснения.

После создания объекта была создана логика для импорта данных из сервиса. Используя встроенный механизм `relations`, связанные объекты загружаются в один запрос. Таким образом, при доступе к проекту вы можете получить всю пользовательскую структуру проекта.

Наконец, модель была протестирована. Добавлены тестовые записи, подтверждена точность создания ссылок и извлечения данных. Данная проверка показала корректность формирования результирующей структуры и соответствие требованиям к ее работе.

Фактический срок реализации задачи составил 3 рабочих дня.

Фрагмент кода, написанного в рамках реализации данной задачи представлен на Рисунках 1.1 и 1.2.

```
@Entity()
export class User {
  @PrimaryGeneratedColumn()
  id: number;

  @Column()
  name: string;

  @ManyToMany(() => Project, project => project.users)
  @JoinTable()
  projects: Project[];
}
```

Рисунок 1.1 – Фрагменты кода сущности User

```
@Entity()
export class Project {
  @PrimaryGeneratedColumn()
  id: number;

  @Column()
  title: string;

  @ManyToMany(() => User, user => user.projects)
  users: User[];
}
```

Рисунок 1.2 – Фрагменты кода сущности Project

Комментарий принимающего лица:

«Связи настроены правильно, данные загружаются корректно. Задача полностью выполнена».

2. Механизм автоматической нормализации и проверки данных

Дата выдачи: 25 сентября 2025 г.

Автор задачи: Насимов Р.Р., руководитель отдела разработки.

Срок для исполнения: 10 рабочих дня.

В системе отсутствуют средства для автоматической обработки поступающих данных от внешних партнеров. Вы должны реализовать функциональность, которая позволит вам принимать JSON-пакеты с информацией о клиентах, проверять обязательные поля (полное имя, адрес электронной почты, номер телефона, источник), нормализовать номера телефонов, проверять правильный и уникальный адрес электронной почты, а также сообщать о количестве успешно обработанных регистраций и ошибках, которые могут быть вызваны. можете указать причину. укажите для каждой недопустимой записи.

Описание выполненных работ:

В рамках работы была проанализирована структура входных данных от партнеров и определены поля, необходимые для обработки: полное имя, адрес электронной почты, телефон, источник. На втором этапе была реализована логика получения и нормализации JSON-пакетов: проверка корректности электронных писем, нормализация телефонных номеров, проверка уникальности данных и ошибок регистрации. Сервис формирует отчет о количестве успешных и ошибочных вводов, позволяя менеджерам оперативно реагировать на неточные данные. После разработки функциональность была протестирована на тестовых данных и была подтверждена корректность обработки и составления отчетов. Работа выполнена качественно, решение полностью соответствует ожидаемым требованиям и функционалу..

Фактический срок реализации задачи составил 2.5 рабочих дня.

Результаты оптимизации представлены на Рисунке 2.1.

```

def process_partner_payload(payload: list) -> dict:
    success = 0
    errors = []

    with transaction.atomic():
        for idx, item in enumerate(payload, start=1):
            try:
                if not all(field in item for field in ("full_name", "email", "phone", "source")):
                    raise ValueError("Отсутствуют обязательные поля")

                validate_email(item["email"])
                phone = normalize_phone(item["phone"])

                if Customer.objects.filter(email=item["email"]).exists():
                    raise ValueError("Email уже существует")

                Customer.objects.create(
                    full_name=item["full_name"],
                    email=item["email"],
                    phone=phone,
                    source=item["source"]
                )

                success += 1

            except (ValidationError, ValueError) as exc:
                errors.append({
                    "index": idx,
                    "data": item,
                    "error": str(exc)
                })

    return {
        "success_count": success,
        "error_count": len(errors),
        "errors": errors[:5] # примеры ошибок
    }

```

Рисунок 2.1 – Фрагмент кода

Комментарий принимающего лица:

«Качественный блогер, который хочет получить доступ к информации, позволяет получать информацию о том, что происходит в интернете. Все это позволяет корректно отправить JSON-файл в Pflichtfelder, адрес электронной почты и телефонный номер указаны как нормальные и действительные. В системе также есть подробный отчет об улучшении условий труда и занятости персонала, для удобства управления и интеграции с внешним партнером erheblich erleichtert. "Ангевандтен" - это метод, основанный на стандартах и практике для решения проблем, возникающих у людей", - говорится в документе. Дас Эрджебнис - это место, где происходят события, связанные с Эрзибартунгом».

3. Реализация JWT-аутентификации и механизма refresh-токенов

Дата выдачи: 15 октября 2025 г.

Автор задачи: Насимов Р.Р., руководитель отдела разработки.

Срок для исполнения: 7 рабочих дней.

Разработать систему аутентификации на основе JWT с двумя токенами (access + refresh). Реализовать маршруты /login и /refresh, обеспечить защиту данных и обновление сессии без повторного входа.

Описание выполненных работ:

На первом этапе я изучил текущую архитектуру на стороне сервера и определил требования к механизму аутентификации (поддержка недолговечных токенов доступа, наличие токенов обновления и возможность безопасного обновления сеансов без повторного входа в систему). В зависимости от требований был выбран подход с использованием библиотеки JSON Web Token и статического хранилища refresh-токенов.

На втором этапе была разработана логика для обработки запроса /login. В рамках процесса сервер генерирует две пары токенов: кратковременный токен (access-токен) доступа (15 секунд) и долговременный токен (refresh-токен) обновления (7 дней). Токен доступа используется для подтверждения запроса, а токен обновления используется для обновления сеанса. После создания токен обновления сохраняется на сервере в виде массива допустимых токенов.

Следующим шагом является обработка пути /refresh. Сервер принимает refresh-токен, проверяет, существует ли токен в хранилище, и проверяет подпись. Для успешной проверки генерируется новый access-токен. Если токен недействителен, сервер корректно возвращает состояние 403.

По окончании операции была выполнена унифицированная проверка пар токенов: успешная авторизация, получение токенов access-токена, корректная выдача новых токенов через /refresh. Логика была протестирована с помощью

Postman, что подтвердило правильность схемы сертификации и соответствие требованиям.

Фактический срок реализации задачи составил 4 рабочих дня.

Фрагмент кода, написанного в рамках реализации данной задачи представлен на Рисунке 3.1.

```
12 ▾ app.post("/login", (req, res) => {  
13   const user = { id: 1, username: "admin" };  
14  
15   const accessToken = jwt.sign(user, ACCESS_SECRET, { expiresIn: "15s" });  
16   const refreshToken = jwt.sign(user, REFRESH_SECRET, { expiresIn: "7d" });  
17  
18   refreshTokens.push(refreshToken);  
19  
20   res.json({ accessToken, refreshToken });  
21 });  
22  
23 ▾ app.post("/refresh", (req, res) => {  
24   const { token } = req.body;  
25  
26   if (!token || !refreshTokens.includes(token)) {  
27     return res.sendStatus(403);  
28   }  
29  
30   try {  
31     const user = jwt.verify(token, REFRESH_SECRET);  
32     const accessToken = jwt.sign(  
33       { id: user.id, username: user.username },  
34       ACCESS_SECRET,  
35       { expiresIn: "15s" }  
36     );  
37     res.json({ accessToken });  
38   } catch (err) {  
39     res.sendStatus(403);  
40   }  
41 });
```

Рисунок 3.1 – Фрагменты кода аутентификации и механизма refresh-токенов

Комментарий принимающего лица:

«Функционал реализован корректно, обновление токена работает стабильно. Предусмотрены ошибки и проверки. Задача выполнена в полном объеме».

4. React-компонент поиска с дебаунсом

Дата выдачи: 10 ноября 2025 г.

Автор задачи: Насимов Р.Р., руководитель отдела разработки.

Срок для исполнения: 5 рабочих дня.

В пользовательском интерфейсе отсутствует оптимизированная поисковая система, что приводит к множеству ненужных вычислений при быстром вводе текста - это замедляет работу приложения. Необходимо разработать компонент React Search с использованием debounce, который будет обрабатывать пользовательский ввод с задержкой и фильтровать данные только после завершения ввода. Решение должно снизить нагрузку на интерфейс и обеспечить бесперебойную работу.

Описание выполненных работ:

Работа началась с анализа требований пользователей: поиск должен работать без задержек для интерфейса, но он не должен отправлять запрос каждые несколько миллисекунд при быстром вводе текста. Для повторения этой задачи был выбран метод разборки с саженкой в 500 мс.

На втором этапе была реализована функция debounce, которая позволяет отложить фильтрацию до тех пор, пока пользователь не перестанет печатать. Информация была встроена в приложение вместо useMemo, что вы избежите от правительства для создания приложения.

Следующим шагом является разработка логики фильтрации данных. В зависимости от значения преобразуется список элементов, который обновляется только по истечении задержки. Это сводит к минимуму количество вычислений и перерисовок интерфейса, что повышает производительность.

На заключительном этапе компонент был протестирован вручную. Были протестированы скрипты для быстрого ввода символов, удаления текста, повторного ввода и проверки на нечувствительность к регистру. Компонент показал стабильную работу, каждый ввод был корректно обработан с задержкой, соответствующей требованиям.

Фактический срок реализации задачи составил 2 рабочих дня.

Результаты оптимизации представлены на Рисунке 4.1.

Введите поисковый запрос:

> а

[0 ms] Ввод принят, ожидание...

> ap

[250 ms] Ввод обновлён, ожидание...

[500 ms] Обработка запроса...

Фильтрация элементов списка...

Результат:

- Apple

Рисунок 4.1 – Результаты оптимизации загрузки

Комментарий принимающего лица:

«Эта функция качественно реализована. Использование `debounds` значительно снизило нагрузку на интерфейс и улучшило отзывчивость приложения. Логика компонентов хорошо продумана и может быть легко расширена. Все пользовательские скрипты обрабатываются правильно. Результат полностью соответствует ожиданиям и требованиям оптимизации».

5. Docker Compose: поднять три сервиса (Node + PostgreSQL + pgAdmin)

Дата выдачи: 25 ноября 2025 г.

Автор задачи: Насимов Р.Р., руководитель отдела разработки.

Срок для исполнения: 15 рабочих дня.

Чтобы упростить разработку и тестирование, не существует централизованного инструмента для запуска всех служб project. Вам необходимо создать конфигурацию Docker Compose, которая настраивает серверную часть, базу данных PostgreSQL и панель управления pgAdmin как отдельные контейнеры. Решение должно включать в себя конфигурации сервисной сети, переадресацию портов и варианты предоставления базы данных, что позволит вместо этого развернуть полноценную среду тестирования.

Описание выполненных работ:

На первом этапе был проведен анализ компонентов, которые необходимо развернуть в контейнерах: сервер Node.js, база данных PostgreSQL и административная панель pgAdmin. Параметры также были определены для обеспечения взаимодействия служб друг с другом, включая единое сетевое пространство Docker.

Затем был разработан файл docker-compose.yml, который описывает три услуги. Для серверной службы было добавлено правило компиляции локального каталога и переадресации портов 3000. Для PostgreSQL устанавливаются переменные среды, необходимые для начальной инициализации базы данных. Служба pgadmin получает имя пользователя и пароль администратора, а также переадресацию портов для доступа к браузеру.

На следующем этапе контейнеры были собраны и спущены на воду. В процессе тестирования были проверены: доступ к API внутреннего сервера, успешное подключение pgAdmin к базе данных, правильность автоматического создания контейнеров и стабильность трех служб одновременно.

В результате инфраструктура была успешно развернута, контейнеры работали стабильно, а взаимодействие между службами осуществлялось корректно. Конфигурация может быть использована в качестве основы для дальнейшей реализации проекта в тестовой среде.

Фактический срок реализации задачи составил 15 рабочих дня.

Результаты оптимизации представлены на Рисунке 5.1.

```
version: "3.9"

services:
  backend:
    build: ./server
    ports:
      - "3000:3000"
    depends_on:
      - db

  db:
    image: postgres:15
    environment:
      POSTGRES_PASSWORD: example
    ports:
      - "5432:5432"

  pgadmin:
    image: dpage/pgadmin4
    environment:
      PGADMIN_DEFAULT_EMAIL: admin@mail.com
      PGADMIN_DEFAULT_PASSWORD: admin
    ports:
      - "5050:80"
```

Рисунок 5.1 – Фрагмент конфигурации

Комментарий принимающего лица:

«Подготовленный Docker-Compose обеспечивает удобный и быстрый запуск всей инфраструктуры в локальной среде. Все службы правильно взаимодействуют друг с другом, конфигурация прозрачна и соответствует передовым методам DevOps. Результат стабилен и может быть использован в качестве основы для дальнейшей автоматизации. Работа ведется на высоком уровне».

ЗАКЛЮЧЕНИЕ

Подводя итоги производственной практики, можно уверенно отметить, что все поставленные задачи были успешно выполнены, а цели практики — достигнуты в полном объёме. Выполнение комплекса технических задач позволило получить всестороннее представление о структуре, организации и реальных рабочих процессах современной IT-компании, а также закрепить знания, полученные в ходе обучения. Практика стала важным шагом в формировании профессионального стиля работы, поскольку позволила не только применить теорию на практике, но и интегрироваться в процессы, типичные для корпоративной разработки.

В рамках практики была выполнена работа над различными аспектами разработки серверной части: реализован функционал массового импорта CSV-данных, создан сервис для обработки JSON-пакетов от внешних партнёров, проведена оптимизация производительности отдельных модулей системы, внедрён механизм WebSocket-уведомлений для обновления данных в реальном времени, а также разработана гибкая система управления ролями и правами доступа. Эти задачи охватывали широкий спектр технологий — от работы с базой данных и построения API до оптимизации SQL-запросов, интеграционных процессов и организации архитектуры приложения.

Полученный опыт работы в команде, участие в планировании, понимание процессов код-ревью и соблюдение регламентов разработки способствовали формированию профессиональной дисциплины и ответственности. Решение прикладных задач в составе настоящего проекта позволило почувствовать значимость технических решений и их влияние на удобство пользователей, производительность системы и бизнес-ценность продукта.

Таким образом, практика стала важным этапом профессионального становления. Она позволила сформировать системный подход к проектированию и реализации функционала, расширить технические компетенции, развить навыки коммуникации и командного взаимодействия.

Полученный опыт служит прочной основой для дальнейшего развития в направлении веб-разработки и формирования полноценного профессионального профиля специалиста в области full-stack разработки.